

Chapter 5: Neural Network and Deep Learning

By Ukesh Thapa

Syllabus

5.1. Neural Networks: Structures and Activation Functions

5.2. Perceptron, Multilayer Perceptron, and Backpropagation

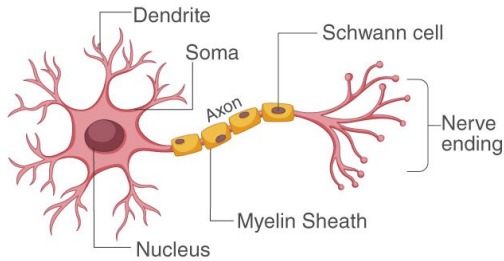
5.3. Introduction to Deep learning

5.4. Concepts on Recurrent and Generative Neural Networks

Total Hours: 6 hrs

Total Marks: 10 Marks (Derivation, Definitions, Explanation)

Biological Neurons



A biological neuron is an electrically excitable cell that processes and transmits information through electrical and chemical signals. A neuron is composed of four primary structures, each serving a specific role in data processing:

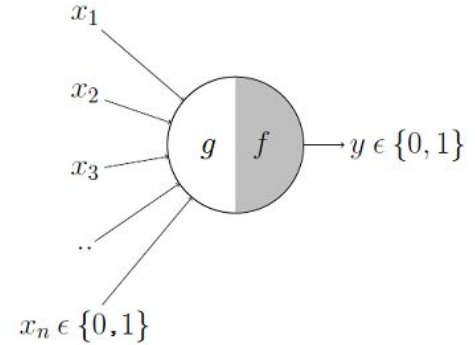
- ❖ **Dendrites** (The Input Receivers)
 - These are branch-like structures that extend from the cell body. Their job is to receive signals from other neurons.
- ❖ **Soma / Cell Body** (The Processor)
 - The soma contains the nucleus and is the "command center." It takes all the incoming signals from the dendrites and combines them.
- ❖ **Axon** (The Transmission Cable)
 - The axon is a long, thin fiber that carries the electrical impulse away from the cell body.
 - To speed up the signal, many axons are insulated by a fatty layer called the **Myelin Sheath**.
- ❖ **Synaptic Terminals** (The Output Transmitters)
 - At the end of the axon are the terminals.
 - These do not actually touch the next neuron.
 - Instead, they release chemicals called neurotransmitters across a tiny gap (the synapse) to trigger the next cell.

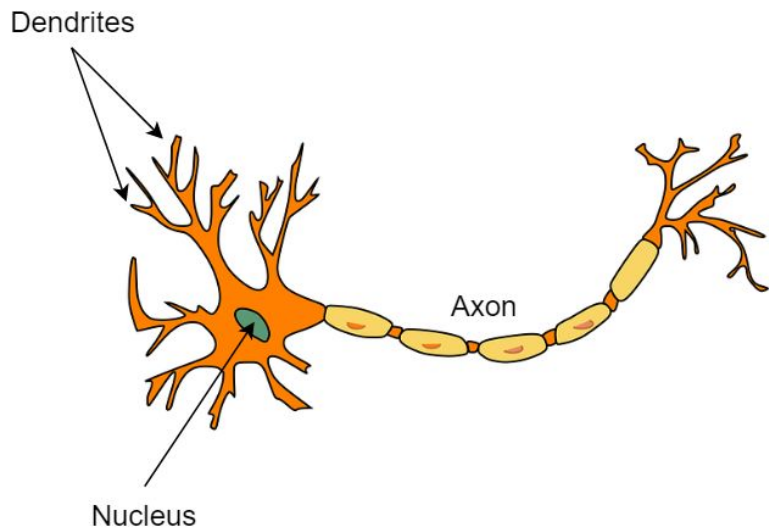
Artificial Neurons

An artificial neuron is a mathematical function that takes inputs, weights them, and decides how much signal to pass forward.

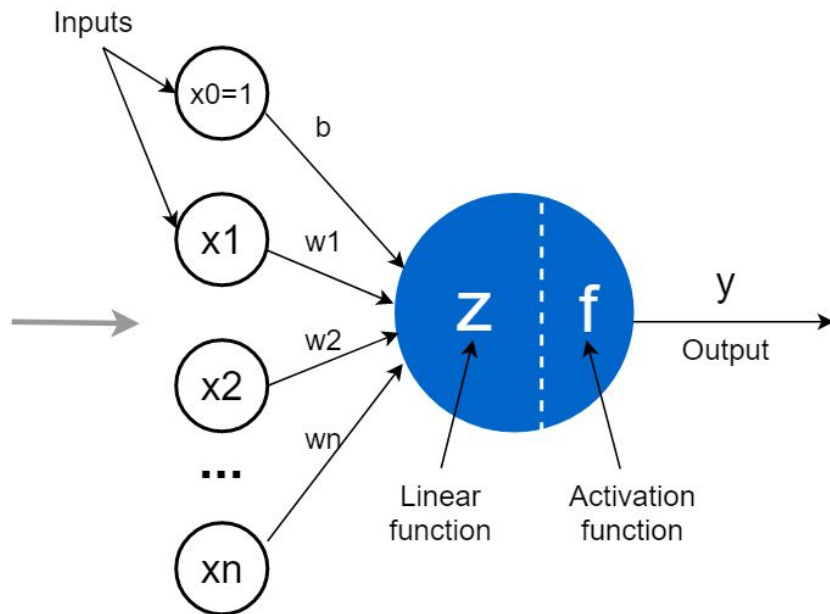
Components of an Artificial Neuron:

- ❖ **Inputs ($x_1, x_2, \dots, x_n$):** These represent the data features. For example, if you are predicting house prices, inputs might be "square footage" and "number of bedrooms."
- ❖ **Weights ($w_1, w_2, \dots, w_n$):** This is the "importance" assigned to each input. During training, the AI adjusts these weights to learn. If an input is very important to the result, its weight will be high.
- ❖ **Bias (b):** An extra value added to the sum. It allows the neuron to shift the activation function up or down, giving the model more flexibility to fit the data.
- ❖ **Activation Function (f):** A mathematical "gatekeeper" that decides whether the neuron should "fire" (and how strongly) based on the calculated sum.





Biological Neuron



Artificial Neuron

Right Brain vs. Left Brain

- ❖ **Left Brain (Symbolic AI):** Logical, linear, analytical, and rule-based. Think of traditional programming (if-then statements).
- ❖ **Right Brain (Neural Networks):** Intuitive, holistic, and pattern-oriented. This is where Deep Learning lives—it "senses" patterns in data rather than following rigid rules.

McCulloch-Pitts (M-P) Neuron

- ❖ The McCulloch-Pitts (M-P) Neuron, proposed in **1943** by neurophysiologist **Warren McCulloch** and **logician Walter Pitts**, is the very first mathematical model of a biological neuron.
- ❖ The M-P neuron was designed to mimic the "all-or-nothing" firing property of biological neurons.
- ❖ The M-P neuron operates in two distinct stages: **Aggregation (Summation)** and **Thresholding**.
- ❖ The main achievement of the M-P model was proving that neurons could perform logical operations.
- ❖ The Components:
 - **Inputs ($x_1, x_2, \dots x_n$):** These are binary signals (0 or 1) coming from other neurons.
 - **Weights (w):** In the original M-P model, weights were not "numbers" like 0.5 or 0.9. They were essentially connections that were either excitatory (+1) or inhibitory (veto).
 - **Threshold (θ):** A fixed value that determines how much signal is needed to fire.
 - **Output (y):** The final binary result.

Mathematical Model of a Neuron (The Perceptron)

A simple neuron calculates a weighted sum of its inputs, adds a bias, and passes it through an activation function. It was invented by **Frank Rosenblatt** in **1958**.

The Perceptron proved that neurons could **learn**.

It shifted the field from "**hard-coded circuits**" to true "**Artificial Intelligence**."

The mathematical representation is:

$$y = f(\sum_{i=1}^n w_i \cdot x_i + b)$$

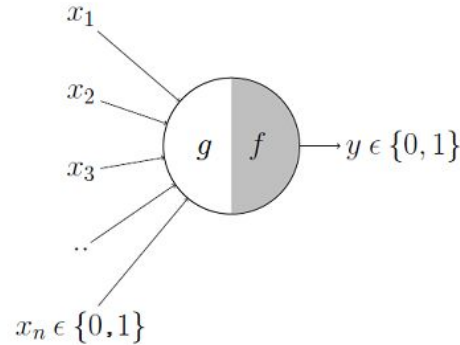
where,

x_i : *Input signals.*

w_i : *Weights (importance of each input).*

b : *Bias (allows the neuron to shift the activation function).*

f : *Activation function.*



Limitation of Perceptron

- ❖ Can Learn Only Linearly Separable Problems
- ❖ Cannot Solve the XOR Problem
- ❖ Uses a Hard Threshold
- ❖ Cannot Learn Complex Nonlinear Relationships
- ❖ Produces Only Binary Decisions
- ❖ Sensitive to Feature Scaling
- ❖ No Hierarchical Representation Learning
- ❖ Convergence Depends on Linearly Separable Data
- ❖ Poor Performance on High-Dimensional Real-World Data

XOR Problem

- ❖ The XOR Problem was famously introduced into the field of AI by Marvin Minsky and Seymour Papert in 1969.
- ❖ They used it as the central mathematical proof in their book Perceptrons to demonstrate the severe limitations of early neural networks.
- ❖ They introduced the XOR (Exclusive OR) function to prove a specific point:

"A single-layer neural network cannot solve any problem that is not linearly separable."

- ❖ AND / OR Gates: These are "Linearly Separable." You can draw a straight line to separate True from False. Perceptrons could solve these easily.
- ❖ XOR Gate: This is the smallest possible logic problem that is Non-Linearly Separable.
- ❖ Why "Hidden Layers" Solve Non-Linearity?
 - Mathematically, this transforms the straight lines into a curve or a parallel track that isolates the middle points from the corner points.

Hebbian Learning

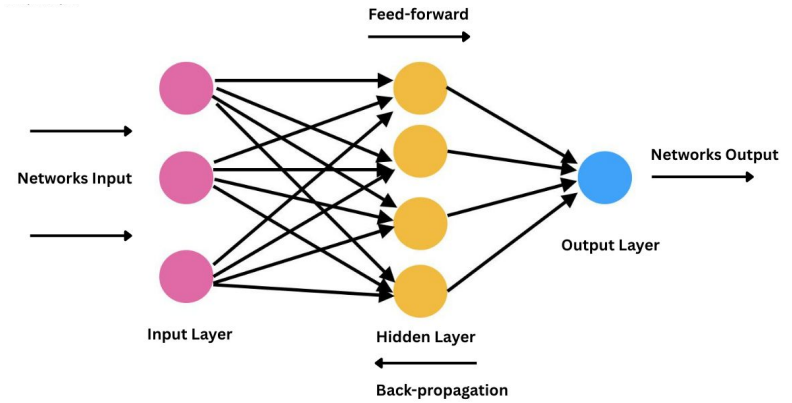
- ❖ Hebbian Learning is one of the oldest and most fundamental learning algorithms in both neuroscience and artificial intelligence. **"Cells that fire together, wire together."**
- ❖ Proposed by Donald Hebb in his 1949 book *The Organization of Behavior*, it explains how neurons adapt and learn by strengthening their connections based on activity.
- ❖ In Artificial Neural Networks, we translate this biological process into a simple equation for updating weights.
 - $w_{new} = w_{old} + n (x \cdot y)$
 - *Where, w is the weight; x the input value; y is the output value; n is the learning rate*
- ❖ Characteristics
 - Unsupervised Learning: The network simply looks for patterns and correlations in the input data itself.
 - Local Learning: The weight update depends only on the two neurons involved (x and y). It doesn't need to know the global error of the whole network.

Hopfield Networks

- ❖ The Hopfield Neural Networks, invented by Dr John J. Hopfield consists of one layer of 'n' fully connected recurrent neurons.
- ❖ It is generally used in performing auto-association and optimization tasks.
- ❖ It is calculated using a converging interactive process and it generates a different response than our normal neural nets.
- ❖ The primary function of a Hopfield Network is Content-Addressable Memory.
 - Standard Memory (RAM): You look up data by its address (e.g., "Give me what is stored at location 0x4F").
 - Associative Memory (Hopfield): You look up data by its content. You give the network a partial or noisy version of a pattern, and it returns the perfect, original pattern.
- ❖ Hopfield used a concept from physics called Energy Minimization.
- ❖ The network is designed so that every time a neuron updates, the Energy (E) decreases or stays the same.
- ❖ It never goes up.
- ❖ Eventually, it hits a minimum and stops. This stable state is the answer.

Neural Network

Mathematically, a Neural Network structure is a series of matrix operations applied sequentially.



❖ The Input Layer

- The input layer is just a vector of numbers representing your data.
- There are no weights or biases here.

❖ The Hidden Layers

- This is where the network structure becomes a Matrix Multiplication.
- These layers extract features and perform the heavy mathematical lifting.
- "Deep Learning" simply means having many hidden layers.

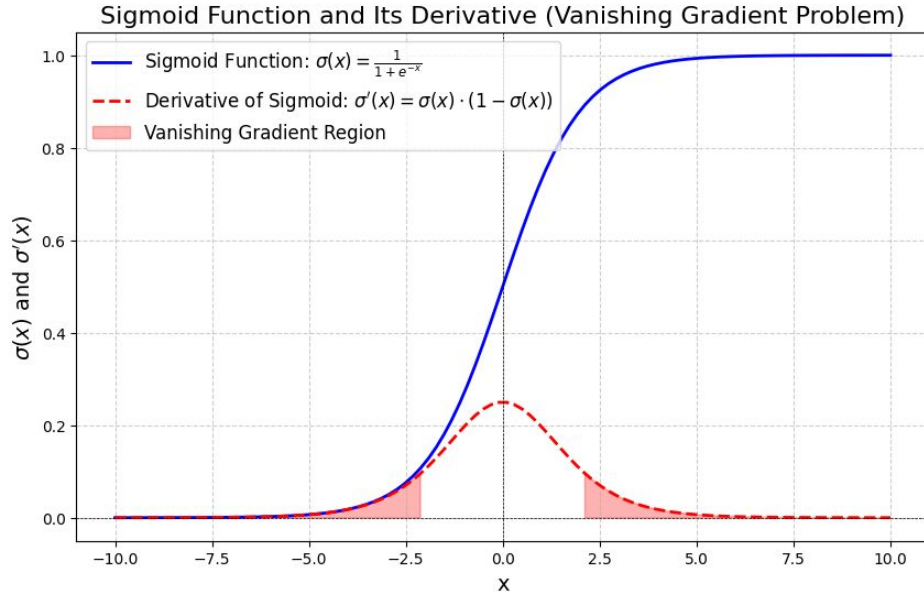
❖ The Output Layer

- The final layer produces the output of the model.
- The format of these outputs varies depending on the specific task like classification, regression.

Activation Function

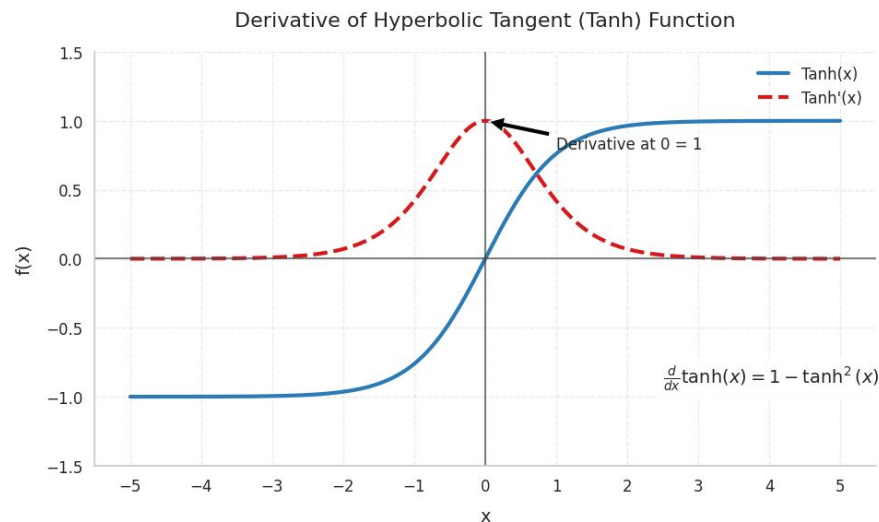
- ❖ Sigmoid:
 - One of the earliest "smooth" activation functions. It squashes any number into a range between 0 and 1
 - Its characteristic "S"-shaped curve makes it particularly useful in scenarios where we need to convert outputs into probabilities.
 - This function is often called the logistic function.
 - Mathematical expression,

$$\sigma = \frac{1}{1+e^{-x}}$$



❖ Tanh:

- Tanh (hyperbolic tangent) is a type of activation function that transforms its input into a value between -1 and 1.
 - As the input becomes more positive, the output approaches 1.
 - As the input becomes more negative, the output approaches -1.
 - At $x = 0$ the output is 0, which is the center of the function.
- It is an S-shaped curve, often referred to as a sigmoid-like curve.
- The tanh function is symmetric around the origin (0), unlike other activation functions like the sigmoid which has a range between 0 and 1.
- Tanh helps mitigate the vanishing gradient problem

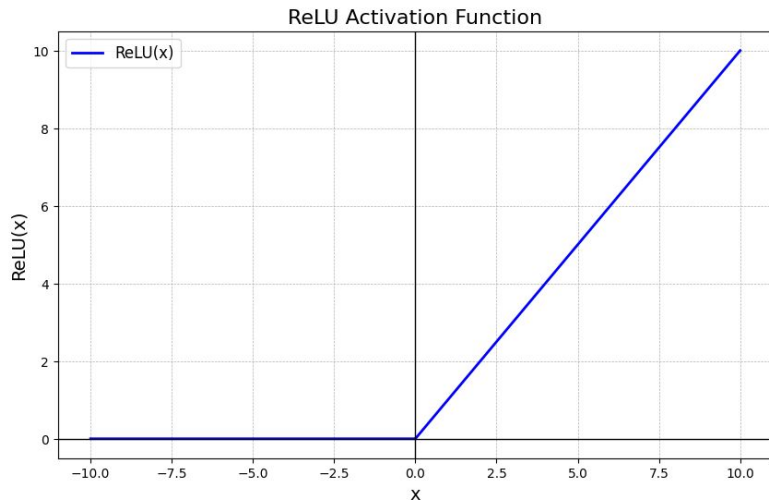


$$f(x) = \tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} = \frac{\sinh(x)}{\cosh(x)}$$

$$\frac{d}{dx} \tanh(x) = 1 - \tanh^2(x)$$

❖ ReLU:

- Rectified Linear Unit (ReLU) is a popular activation functions used in neural networks, especially in deep learning models.
- It has become the default choice in many architectures due to its simplicity and efficiency.
- The ReLU function is a piecewise linear function that outputs the input directly if it is positive; otherwise, it outputs zero.
- ReLU allows positive values to pass through unchanged while setting all negative values to zero.
- learn more complex data patterns and model intricate relationships between features.
- Only a fraction of neurons activate at any given time. This can lead to more efficient and faster computation.
- ReLU offers computational advantages in terms of backpropagation, as its derivative is simple—either 0 (when the input is negative) or 1 (when the input is positive)



Mathematically,

$$f(x) = \max(0, x)$$

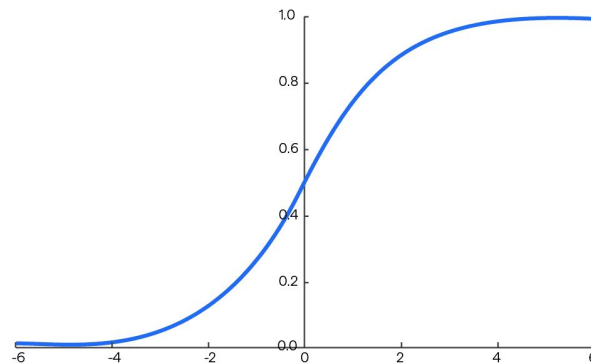
❖ Softmax:

- In Deep Learning, activation functions are important because they introduce non-linearity into neural networks allowing them to learn complex patterns.
- Softmax Activation Function transforms a vector of numbers into a probability distribution, where each value represents the likelihood of a particular class.
- It is especially important for multi-class classification problems.

- Each output value lies between 0 and 1.
- The sum of all output values equals 1.

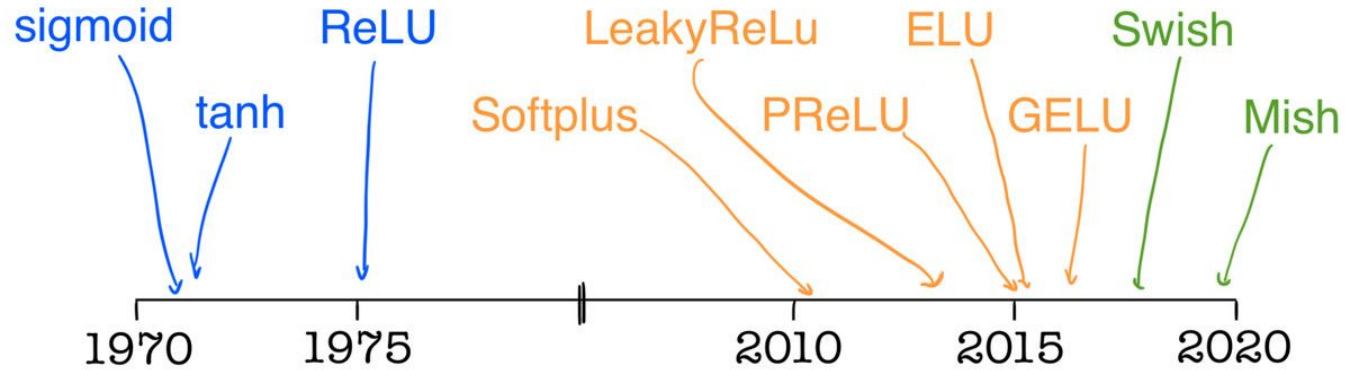
➤ Characteristics

- Normalization: Converts logits into a probability distribution where the sum equals 1.
- Exponentiation: Amplifies larger values making the model's confidence more pronounced.
- Differentiable: Enables gradient-based optimization during backpropagation.
- Probabilistic Interpretation: Makes output easier to interpret as class likelihoods.



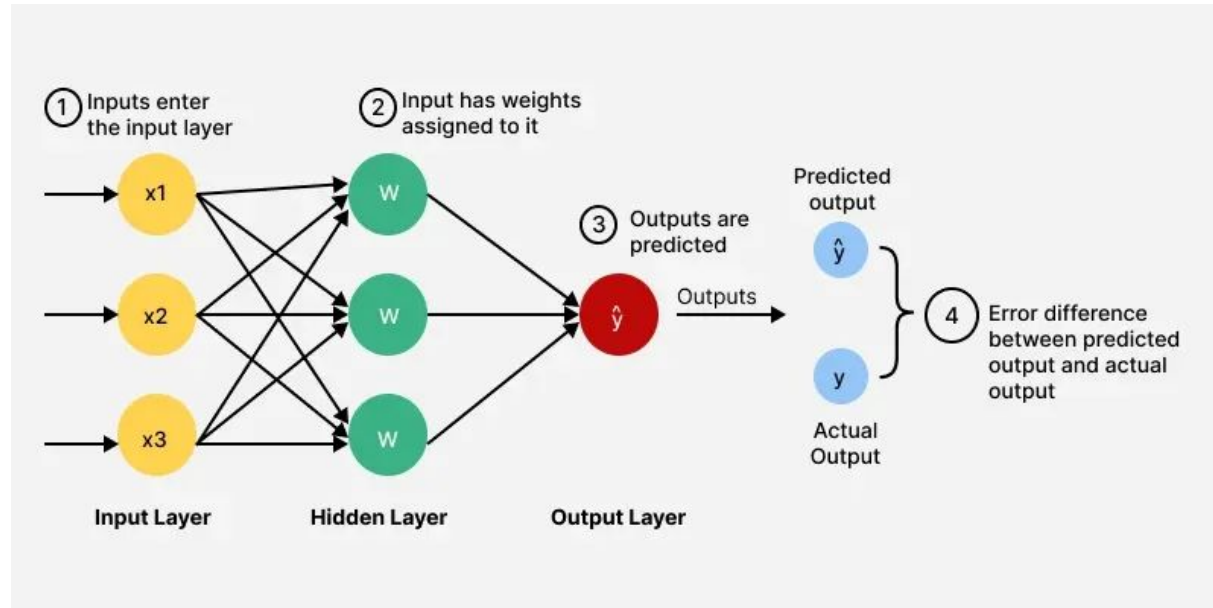
$$\sigma(\vec{z})_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

Activation Function



Feed Forward Neural Network

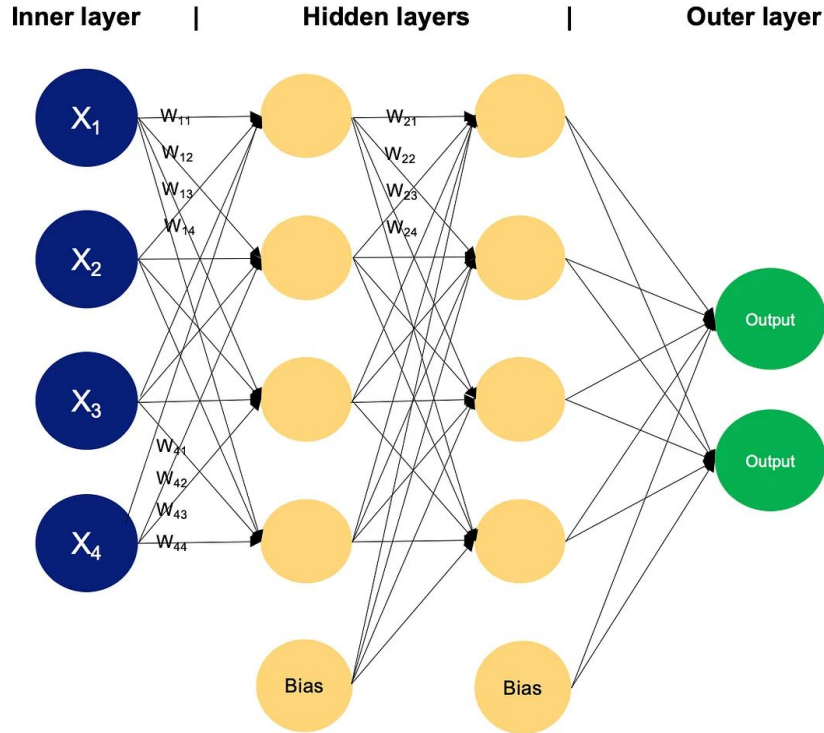
- ❖ **Artificial Neural Network** in which information flows in a single direction.
- ❖ It is mainly used for pattern recognition tasks like image and speech classification.



Structure of a Feedforward Neural Network

- ❖ **Input Layer:** The input layer consists of neurons that receive the input data. Each neuron in the input layer represents a feature of the input data.
- ❖ **Hidden Layers:** One or more hidden layers are placed between the input and output layers. These layers are responsible for learning the complex patterns in the data. Each neuron in a hidden layer applies a weighted sum of inputs followed by a non-linear activation function.
- ❖ **Output Layer:** The output layer provides the final output of the network. The number of neurons in this layer corresponds to the number of classes in a classification problem or the number of outputs in a regression problem.

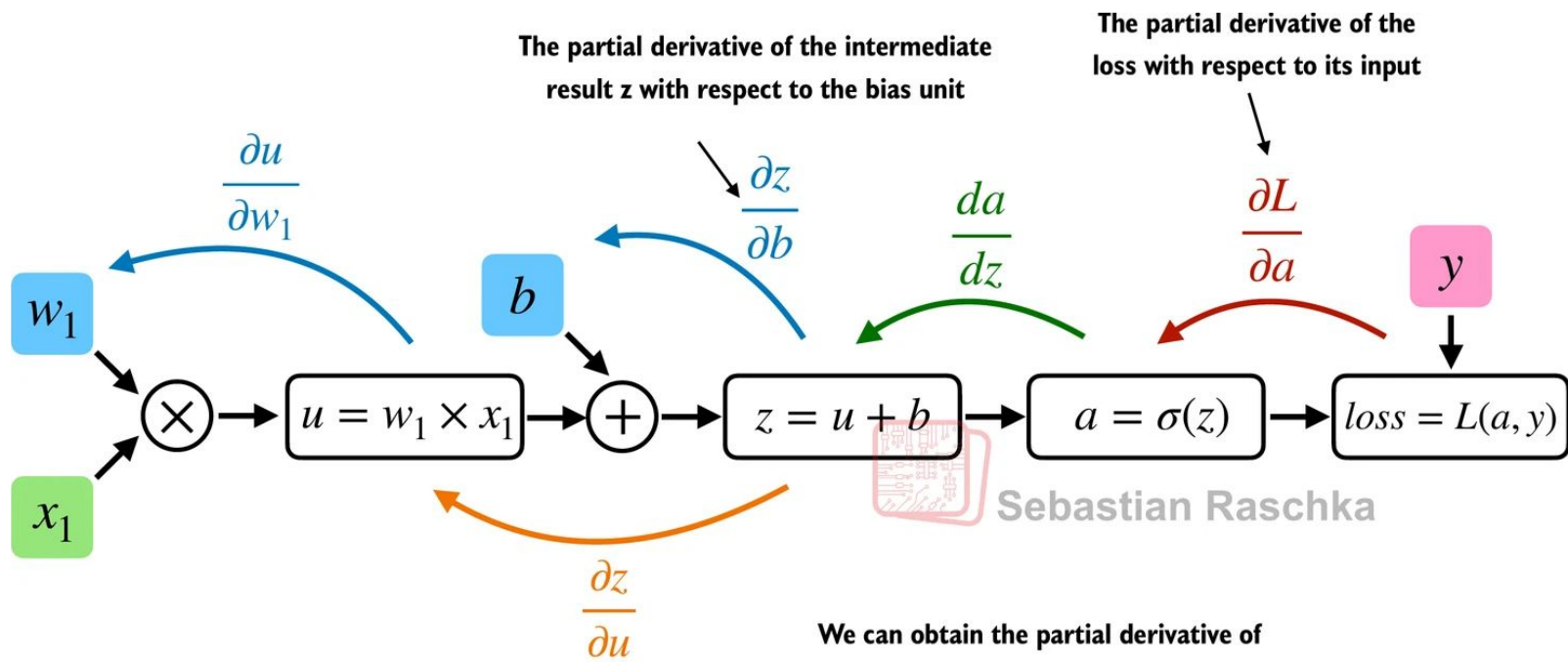
Multilayer Perceptron Architecture



Function Multilayer Perceptron

- ❖ An MLP acts as a universal function approximator.
- ❖ Primary Function:
 - Classification: Assigning input data into distinct categories (e.g., spam vs. non-spam, image category prediction) using non-linear decision boundaries.
 - Regression: Predicting continuous numerical values (e.g., house prices, stock trends).
 - Feature Representation: Transforming complex, non-linearly separable raw data into high-dimensional representations where patterns become linearly separable.

Backpropagation



$$\frac{\partial L}{\partial w_1} = \frac{\partial u}{\partial w_1} \times \frac{\partial z}{\partial u} \times \frac{da}{dz} \times \frac{\partial L}{\partial a}$$

We can obtain the partial derivative of the loss with respect to the trainable weight by chaining the individual partial derivative in the graph

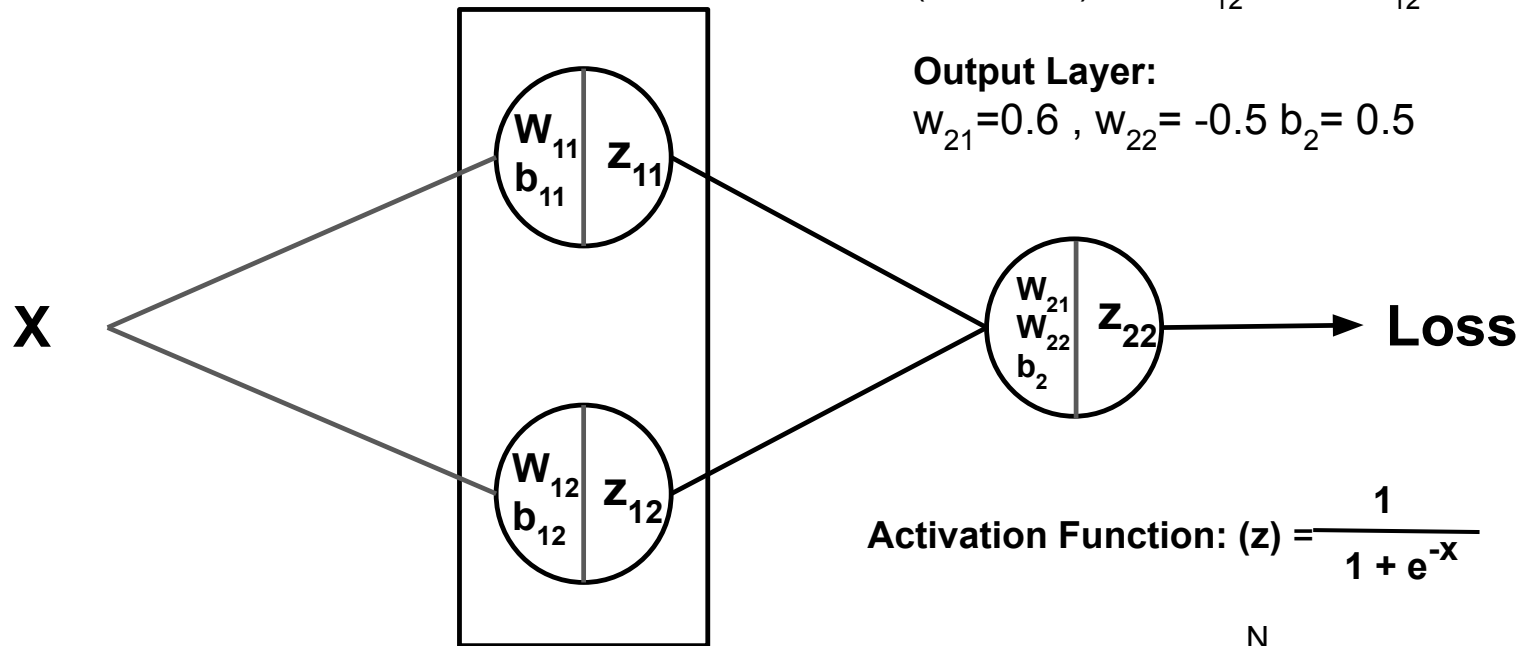
$$\frac{\partial L}{\partial b} = \frac{\partial z}{\partial b} \times \frac{da}{dz} \times \frac{\partial L}{\partial a}$$

Similar to above, we can compute the partial derivative of the trainable derivative by applying the chain rule

Input (X) = 0.5

Assume:

True label = 1



Initial Parameters:

Hidden Layer

(Neuron 1): $w_{11}=0.8$, $b_{11}= 0.2$

(Neuron 2): $w_{12}=-0.4$, $b_{12}= 0.5$

Output Layer:

$w_{21}=0.6$, $w_{22}= -0.5$ $b_2= 0.5$

Activation Function: (z) =
$$\frac{1}{1 + e^{-x}}$$

Binary Cross Entropy:
$$\sum_{i=1}^N [\hat{y} \log(\hat{y}) + (1-\hat{y}) \log(1-\hat{y})]$$

Hidden Layer 1

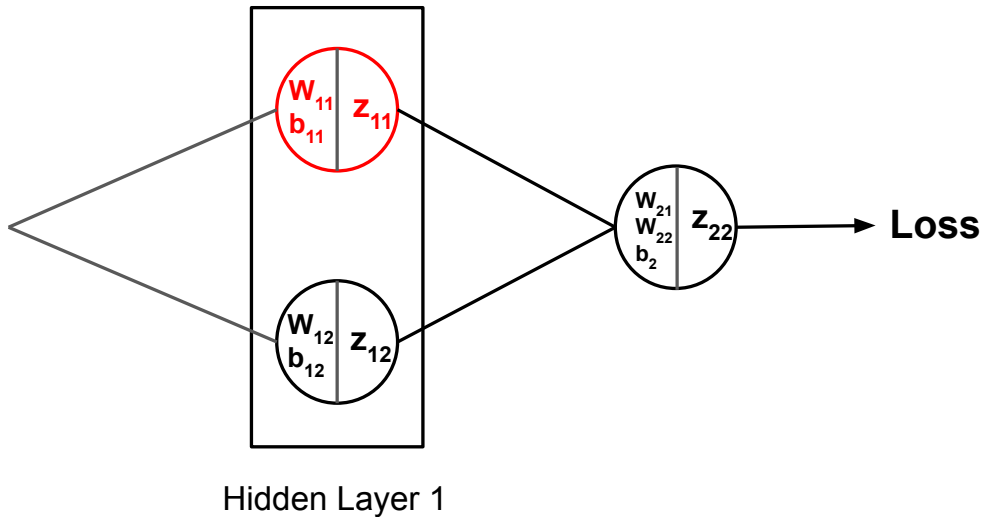
Forward Pass

Neuron 1:

$$z_{11} = w_{11} x + b_{11} = 0.8 * 0.5 + 0.2 = 0.6$$

$$h_1 = \sigma(z_{11}) = \frac{1}{1 + e^{-0.600}} \approx 0.6457$$

x



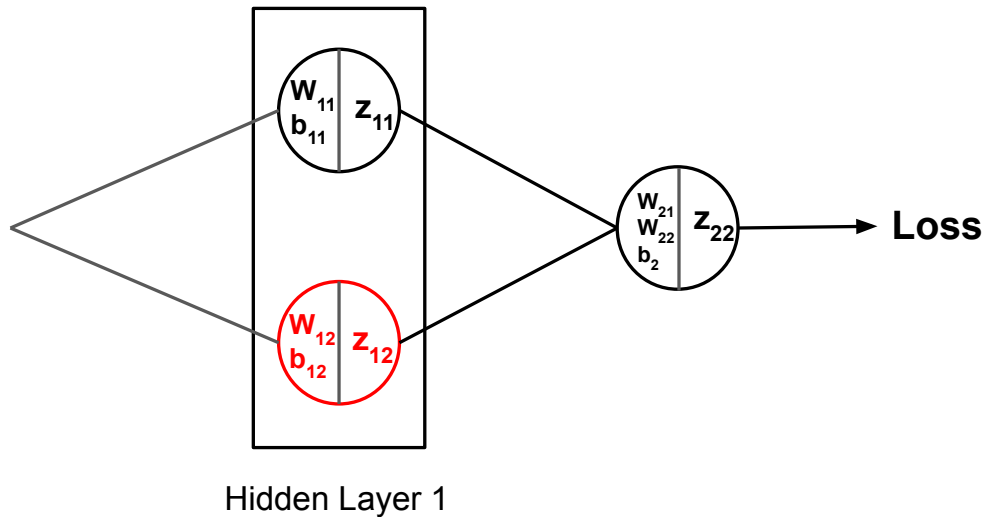
Forward Pass

Neuron 2:

$$z_{12} = w_{12}x + b_{12} = -0.4 * 0.5 + 0.5 = 0.3$$

$$h_2 = \sigma(z_{12}) = \frac{1}{1 + e^{-0.300}} \approx 0.5744$$

x



Forward Pass

Output Layer:

$$z_2 = w_{21}h_1 + w_{22}h_2 + b_2$$

$$\begin{aligned} z_2 &= (0.6)(0.6457) + (-0.5)(0.5744) + 0.1 \\ &= 0.3874 - 0.2872 + 0.1 = 0.2002 \end{aligned}$$

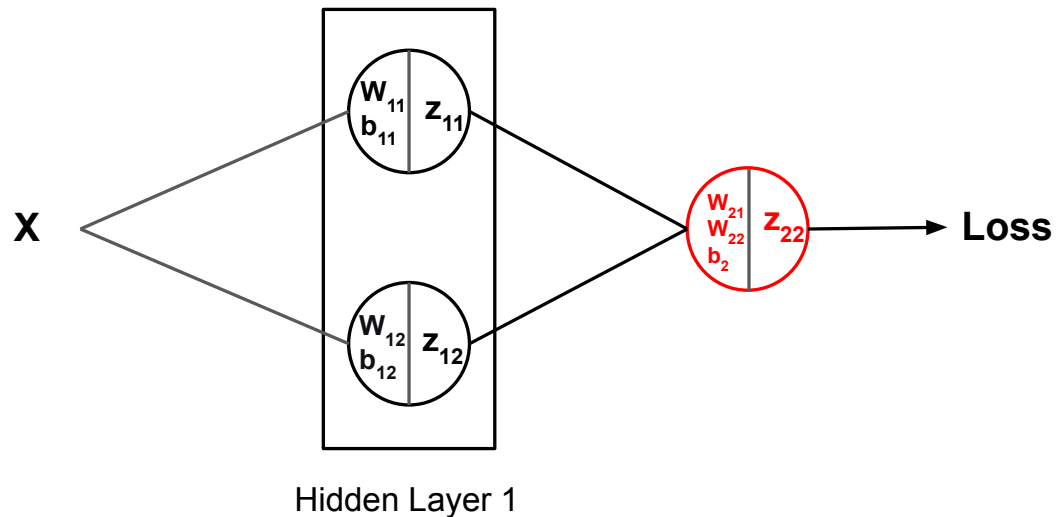
$$h_2 = \sigma(z_{12}) = \frac{1}{1 + e^{-0.2002}} \approx 0.5499$$

Compute loss

$$y = 1,$$

$$\text{the loss } L = -\log(\hat{y})$$

$$= -\log(0.5499) \approx 0.5981$$



Backward Pass

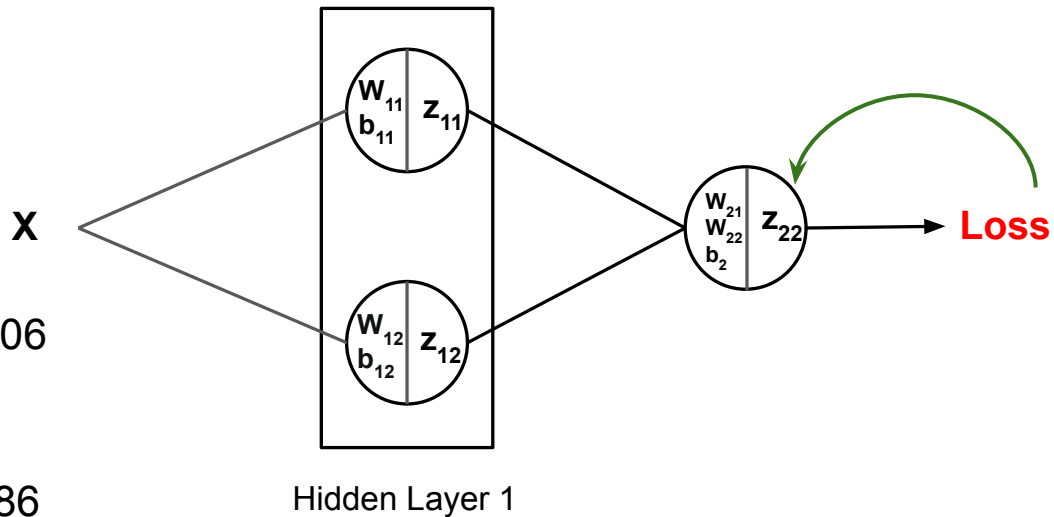
Step A:

$$\frac{\partial L}{\partial z_2} = \hat{y} - y = 0.5499 - 1 = -0.4501$$

$$\frac{\partial L}{\partial w_{21}} = \frac{\partial L}{\partial z_2} \cdot h_1 = (-0.4501)(0.6457) \approx -0.2906$$

$$\frac{\partial L}{\partial w_{22}} = \frac{\partial L}{\partial z_2} \cdot h_2 = (-0.4501)(0.5744) \approx -0.2586$$

$$\frac{\partial L}{\partial b_2} = \frac{\partial L}{\partial z_2} \cdot 1 = -0.4501$$

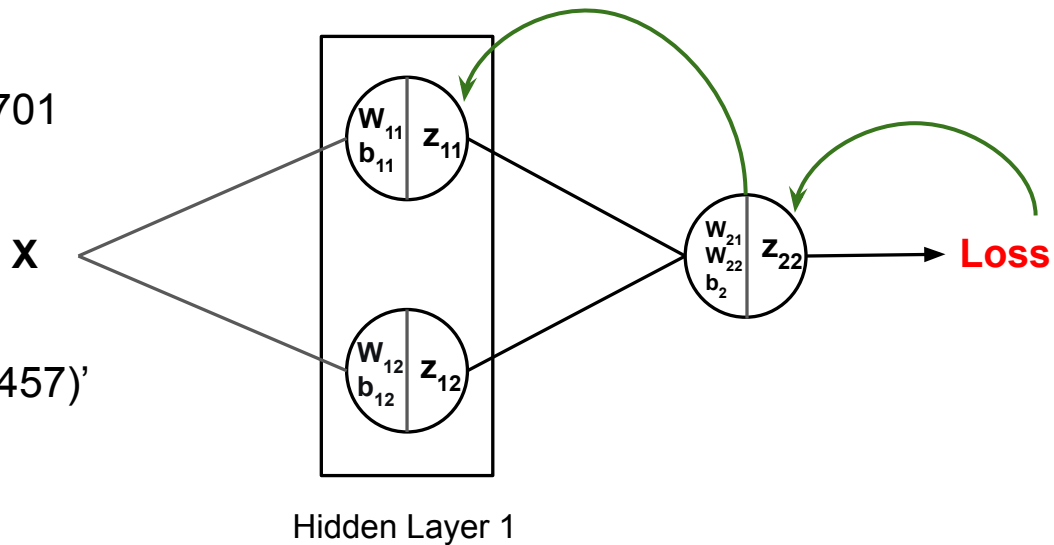


Backward Pass

Hidden Neuron 1:

$$\frac{\partial L}{\partial h_1} = \frac{\partial L}{\partial z_2} \cdot w_{21} = -0.4501 * 0.6 = -0.2701$$

$$\begin{aligned} \frac{\partial L}{\partial z_{11}} &= \frac{\partial L}{\partial h_1} \cdot h_1(1 - h_1) \\ &= -0.2701 * 0.6457 * (1 - 0.6457)' \\ &\approx -0.0618 \end{aligned}$$



Backward Pass

Hidden Neuron 1:

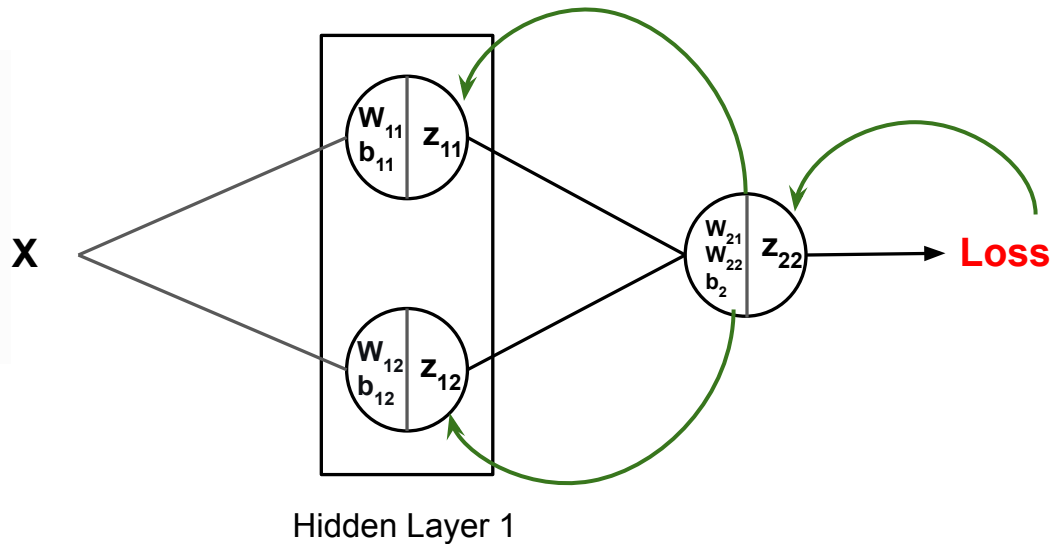
$$\frac{\partial L}{\partial w_{11}} = \frac{\partial L}{\partial z_{11}} \cdot x = (-0.0618)(0.5) \approx -0.0309$$

$$\frac{\partial L}{\partial b_{11}} = \frac{\partial L}{\partial z_{11}} \cdot 1 \approx -0.0618$$

Hidden Neuron 2:

$$\frac{\partial L}{\partial h_2} = \frac{\partial L}{\partial z_2} \cdot w_{22} = (-0.4501)(-0.5) = 0.2251$$

$$\frac{\partial L}{\partial z_{12}} = \frac{\partial L}{\partial h_2} \cdot h_2(1 - h_2) = (0.2251) \cdot (0.5744)(1 - 0.5744) \approx 0.05$$

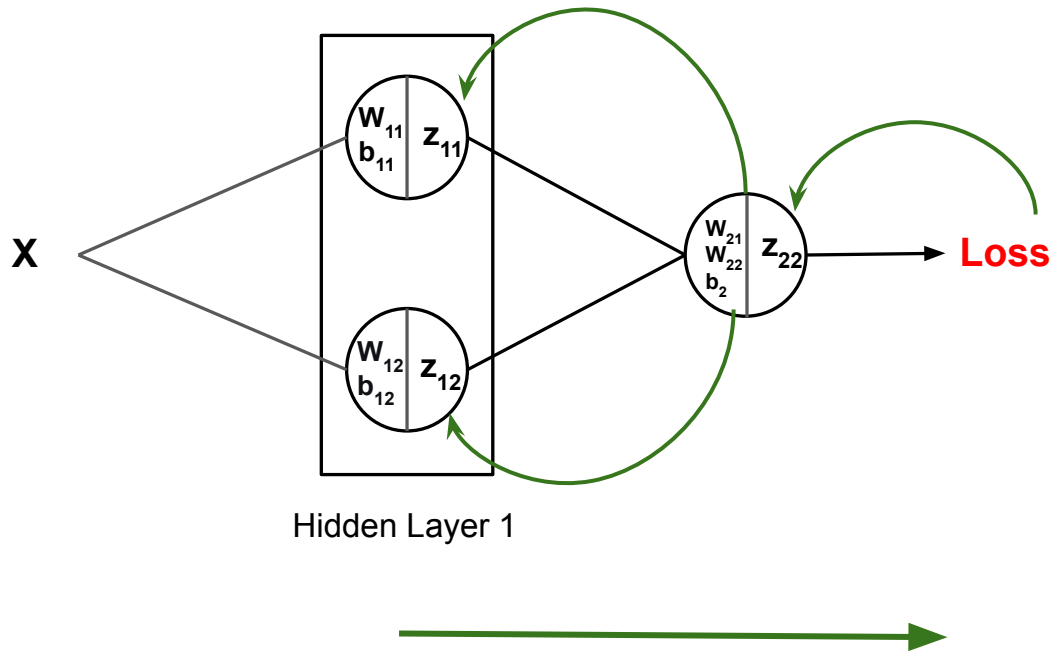


Backward Pass

Hidden Neuron 2:

$$\frac{\partial L}{\partial w_{12}} = \frac{\partial L}{\partial z_{12}} \cdot x = (0.0550)(0.5) \approx 0.0275$$

$$\frac{\partial L}{\partial b_{12}} = \frac{\partial L}{\partial z_{12}} \cdot 1 \approx 0.0550$$



Machine Learning vs Deep Learning

Feature	Machine Learning	Deep Learning
Feature Extraction	Requires manual, domain-expert feature engineering (e.g., SIFT, HOG, hand-picked tabular stats).	Automatically discovers feature hierarchies directly from raw data.
Data Scaling	Performance plateaus once data volume exceeds a certain threshold.	Performance continues to improve as data scale and model size increase.
Computational Needs	Runs efficiently on standard CPUs.	Requires specialized hardware (GPUs/TPUs) for parallel matrix operations.
Interpretability	Often easier to interpret (e.g., Decision Trees, Linear Regression).	Functions largely as a "black box" due to high-dimensional non-linear interactions.

Why Deep Representations?

Deep architectures are exponentially more efficient at representing complex functions than shallow, wide networks.

- ❖ **Hierarchical Abstraction:** In visual processing, Layer 1 detects basic edges; Layer 2 combines edges to form textures and shapes; Layer 3 forms object parts (eyes, wheels); Layer 4 detects full objects.
- ❖ **Circuit Theory Argument:** Formally, computing certain mathematical functions requires an exponential number of hidden units in a shallow 1-hidden-layer network, but can be computed using only a polynomial number of units if the network is deep.

Prerequisites for Successful Deep Learning

- ❖ **Sufficient High-Quality Data:** Deep networks have high capacity and overfit easily on small or noisy datasets.
- ❖ **Computational Infrastructure:** Access to accelerated hardware (GPUs/TPUs) with high memory bandwidth.
- ❖ **Proper Weight Initialization:** Using techniques like He initialization (for ReLU) or Xavier/Glorot initialization (for Tanh) to prevent vanishing or exploding gradients at step zero.
- ❖ **Data Normalization:** Scaling inputs (e.g., standardizing features to zero mean and unit variance) to ensure balanced optimization landscapes.
- ❖ **Regularization Strategies:** Applying Dropout, L_2 weight decay, or Data Augmentation to ensure generalization to unseen data.

Convolution Neural Network (CNN)

❖ The Problem:

- Standard Network, take a small photo (200 x 200) pixels into a flatten row of 40,000 pixels. The Network loses the 2D shape of image
- We need billions of parameters for hidden layer connection, it's inefficient and slow.

❖ The Solution

- We need network that looks at patches of the image (like eye do) rather than whole thing at once.

❖ CNN

- A specialized Neural Network designed for processing data with a grid like topology
- Example:
 - 1D Grid: Time series data (Sample taken at regular interval)
 - 2D Grid: Image data (pixels arranged in a grid)

Convolution Neural Network (CNN)

❖ Convolution vs Matrix Multiplication

- **Standard Network:** Use simple matrix multiplication (every input unit interact with every unit output)
- **CNN :** Use a mathematical Operation called Convolution

❖ Biological Inspiration

- CNNs are a prime example of Neuroscientific principles influencing deep learning
- Emulate how brain works

❖ The convolution Operation

- How can we understand convolution?
 - A signal processing problem: Smoothing Noise

Recurrent Neural Network (RNN)

❖ Problem

- The number of times any word repeated it actually cannot understand or relate the word.
 - Example: A cat is sleep, it is on the mat.
 - What does “it” refers to its cat or the mat
- The **Feed Forward Neural Network**, requires massive number of parameter to relearn the rules. It cannot handle sequence of varying length

❖ Recurrent Neural Network

- A class of neural networks designed to process sequential data by retaining information from previous steps.
- 1d-convolution: Shallow sharing
 - Although it share the parameter they are shallow in the time dimension
- In RNN the output of a state at time t is depend on the state at $t-1$ which itself depend on the $t-2$.

Note:

A deep computational graph while convolution only look for the local snapshot

RNN retain memory of entire history of the sequence upto that point

RNN

❖ Equation

$$s^{(t)} = f(s^{(t-1)}, \Theta)$$

Where,

S^t is state of the system at time t ,

Θ is the set of parameter (weight and biases) which does not change with time

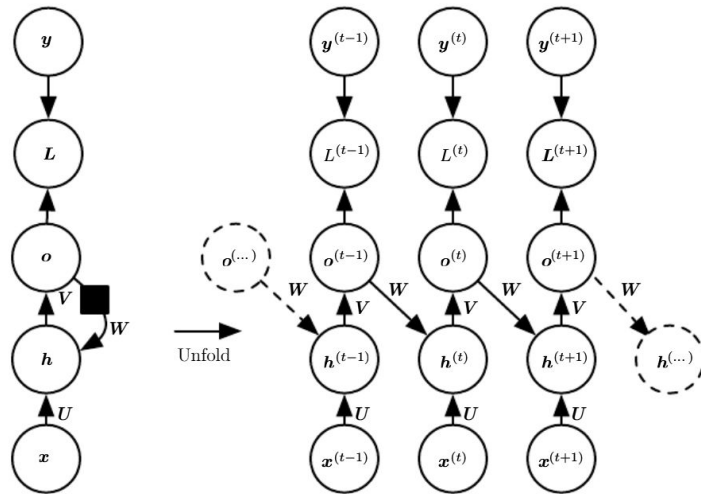
This equation is recurrent because My current state is determined by previous state

RNN Architecture

Where the parameters are the bias vectors $\mathbf{b}$ and $\mathbf{c}$ along with the weight matrices $\mathbf{U}, \mathbf{V}$ and $\mathbf{W}$, respectively

The total loss for a given sequence of $\mathbf{x}$ values paired with a sequence of $\mathbf{y}$ values would then be just the sum of the losses over all the time steps.

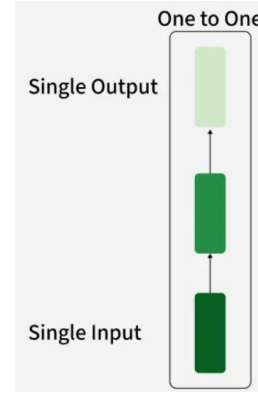
$$\begin{aligned} \mathbf{a}^{(t)} &= \mathbf{b} + \mathbf{W}\mathbf{h}^{(t-1)} + \mathbf{U}\mathbf{x}^{(t)}, \\ \mathbf{h}^{(t)} &= \tanh(\mathbf{a}^{(t)}), \\ \mathbf{o}^{(t)} &= \mathbf{c} + \mathbf{V}\mathbf{h}^{(t)}, \\ \hat{\mathbf{y}}^{(t)} &= \text{softmax}(\mathbf{o}^{(t)}), \end{aligned}$$



Types Of Recurrent Neural Networks

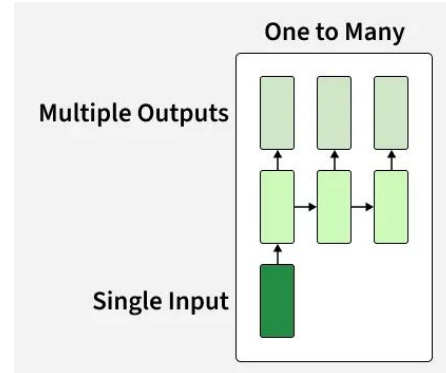
❖ One-to-One RNN

- This is the simplest type of neural network architecture where there is a single input and a single output.
- It is used for straightforward classification tasks such as binary classification where no sequential data is involved.



❖ One-to-Many RNN

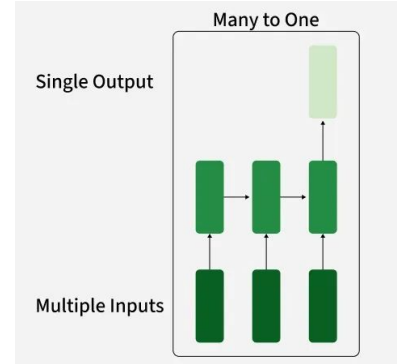
- In a One-to-Many RNN the network processes a single input to produce multiple outputs over time.
- This is useful in tasks where one input triggers a sequence of predictions (outputs).



Types Of Recurrent Neural Networks

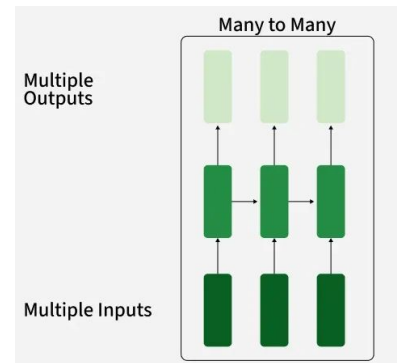
❖ Many-to-One RNN

- The Many-to-One RNN receives a sequence of inputs and generates a single output.
- This type is useful when the overall context of the input sequence is needed to make one prediction.



❖ Many-to-Many RNN

- The Many-to-Many RNN type processes a sequence of inputs and generates a sequence of outputs.



Variants of Recurrent Neural Networks (RNNs)

- ❖ Vanilla RNN
- ❖ Bidirectional RNNs
- ❖ Long Short-Term Memory Networks (LSTMs)
- ❖ Gated Recurrent Units (GRUs)

History of Generative Models

❖ Statistical Foundations (Pre-2000s):

- Early models relied heavily on probability distributions.
- Markov chains, Naive Bayes, and Gaussian Mixture Models (GMMs) were used to estimate underlying data patterns.

❖ Energy-Based & Graphical Models (2000s):

- Models like Restricted Boltzmann Machines (RBMs) and Hidden Markov Models (HMMs) gained popularity for unsupervised learning, speech recognition, and collaborative filtering.

❖ The Deep Learning Revolution (2014–Present):

- 2014: Variational Autoencoders (VAEs) and Generative Adversarial Networks (GANs) were introduced, completely changing computer vision.
- 2017: The Transformer architecture transformed Natural Language Processing (NLP) with autoregressive models (e.g., early GPT models).
- 2020s: Diffusion Models (like Stable Diffusion) emerged, taking over state-of-the-art image and video generation.

Generative Adversarial Networks (GANs)

- ❖ Introduced by Ian Goodfellow et al. in 2014
- ❖ the generation process as a game between two competing neural networks:
 - the Generator and
 - the Discriminator.
- ❖ It solves a two-player zero-sum game between two networks
 - Generator (G): A parametric function $G(z; \theta_g)$ mapping latent noise $z \sim p_z$ to the data space x .
 - Discriminator (D): A parametric function $D(x; \theta_d)$ mapping a data sample x to a scalar probability $[0, 1]$ indicating whether x is real.
- ❖ The minimax game is defined as:

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{\text{data}}(x)} [\log D(x)] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z)))]$$

GAN

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{\text{data}}(x)} [\log D(x)] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z)))]$$

$\mathbf{p}_{\text{data}}(\mathbf{x})$: The true distribution of real data (e.g., real images).

$\mathbf{p}_z(\mathbf{z})$: A simple prior distribution for input noise (usually standard Gaussian distribution $\mathcal{N}(0, I)$).

$\mathbf{G}(\mathbf{z})$: The **Generator**, a function parameterized by neural network weights that maps random noise vector z to a synthetic sample space $G(z)$.

$\mathbf{D}(\mathbf{x})$: The **Discriminator**, a function that outputs a single scalar probability in $[0, 1]$ predicting whether x came from real data (1) or synthetic data (0).

$\mathbb{E}_{\mathbf{x} \sim \mathbf{p}_{\text{data}}(\mathbf{x})}[\cdot]$: The **Expected Value** over all real samples drawn from the true data distribution.

$\mathbb{E}_{\mathbf{z} \sim \mathbf{p}_z(\mathbf{z})}[\cdot]$: The **Expected Value** over all random noise vectors sampled from the noise distribution.

Application of GAN

- ❖ Image-to-Image Translation
- ❖ Image Super-Resolution
- ❖ Medical Image Augmentation & Privacy
- ❖ Synthetic Data Generation for Computer Vision